

UNIVERSIDADE FEDERAL DE PERNAMBUCO
CENTRO DE INFORMÁTICA

CIN0135 — Estruturas de Dados Orientadas a Objetos
Professor: Francisco Paulo Magalhães Simões

SISTEMA DE GESTÃO DE CONDOMÍNIO

Relatório do Projeto Prático

Equipe

Beatriz Luna
Clécio Muniz
Sâmia Freitas
Vivian Azevedo

Recife, outubro de 2026

1. Introdução

Este relatório descreve o Sistema de Gestão de Condomínio, desenvolvido pelo nosso grupo como Projeto Prático da disciplina CIN0135. O objetivo da disciplina é oferecer uma primeira experiência com Programação Orientada a Objetos (POO) na linguagem C++. Por isso, o foco do trabalho não foi apenas fazer um sistema funcionar, mas organizá-lo em classes, aplicar herança, polimorfismo, encapsulamento e design patterns, e explicar como cada conceito foi usado.

Escolhemos a opção 1 do enunciado, que é construir um sistema de informação com CRUD (criar, consultar, atualizar e remover dados) conectado a um banco de dados. O tema é um condomínio residencial. Ele foi escolhido porque é uma situação do dia a dia, com entidades fáceis de identificar (apartamentos, moradores, visitantes, funcionários, áreas comuns e reservas) e regras de negócio claras, como evitar duas reservas no mesmo horário ou permitir que apenas o porteiro registre visitas.

O sistema tem um núcleo em C++17, que não depende de interface, e duas formas de uso: um menu de terminal e uma interface gráfica feita com Qt 6. Os dados ficam em um banco SQLite. O projeto usa CMake para compilar e Git e GitHub para o trabalho em equipe.

Tabela 1 – Dados gerais do projeto

Item	Descrição
Disciplina	CIN0135 — Estruturas de Dados Orientadas a Objetos (CIn-UFPE, 2026.2)
Professor	Francisco Paulo Magalhães Simões
Equipe	Vivian Azevedo, Clécio Muniz, Sâmia Freitas e Beatriz Luna
Linguagem	C++17
Banco de dados	SQLite (código incluído no próprio repositório, em third_party/sqlite)
Interface gráfica	Qt 6 Widgets (bônus)
Build	CMake e CTest
Repositório	github.com/vivianazevedo/edoo-adm-condominio
Página do projeto	vivianazevedo.github.io/edoo-adm-condominio

Ficaram fora do escopo: pagamento real, login e senha, uso simultâneo por vários usuários e versão web ou mobile. O porteiro, por exemplo, é identificado escolhendo um funcionário com cargo de porteiro no formulário, sem autenticação.

2. Descrição do sistema

O sistema atende dois tipos de usuário: o administrador do condomínio, que cadastra e mantém os dados, e o porteiro, que registra as visitas. Os moradores são cadastrados no sistema, mas não fazem login: as reservas são feitas escolhendo o morador na tela.

2.1 Funcionalidades

Tabela 2 – Funcionalidades por módulo

Módulo	O que o usuário consegue fazer
Apartamentos	Cadastrar, listar, editar e remover apartamentos (bloco, número e andar)
Moradores	Cadastrar, listar, editar e remover moradores, sempre vinculados a um apartamento
Funcionários	Cadastrar, listar, editar e remover funcionários, com cargo, turno e data de admissão
Áreas comuns	Cadastrar, listar, editar e remover salão de festas, piscina e churrasqueira
Reservas	Criar uma reserva, cancelar e consultar por morador, por área e data
Visitas	Cadastrar visitante, registrar entrada e saída (só pelo porteiro) e consultar o histórico por apartamento

2.2 Histórias de usuário

Tabela 3 – Histórias de usuário do MVP

ID	História	Critério de aceite
US01	Como administrador, quero cadastrar, listar, editar e remover apartamentos	Bloco e número únicos; não remove apartamento com moradores
US02	Como administrador, quero cadastrar e gerenciar moradores vinculados a um apartamento	CPF válido e único; apartamento obrigatório
US03	Como administrador, quero cadastrar áreas comuns	Capacidade maior que zero; abertura antes do fechamento
US04	Como morador, quero reservar uma área para uma data e horário	Valida funcionamento, capacidade e conflito; calcula o valor
US05	Como morador, quero cancelar uma reserva	Só com 24 horas de antecedência; o horário volta a ficar livre
US06	Como morador, quero consultar reservas por morador, área e data	Lista com os dados da reserva
US07	Como porteiro, quero registrar visitantes e suas entradas e saídas	Só cargo Porteiro; saída depois da entrada; histórico preservado
US08	Como administrador, quero cadastrar e gerenciar funcionários	Cargo e turno obrigatórios
US09	Como usuário, quero fazer tudo isso por telas	Cada CRUD acessível na interface; erros em diálogo claro

2.3 Regras de negócio

As regras ficam nos serviços e nas classes do modelo, nunca na interface. Quando uma regra é quebrada, o sistema lança uma exceção que o terminal imprime e a interface Qt mostra em uma janela de aviso.

Tabela 4 – Regras de negócio e onde são aplicadas

ID	Regra	Onde é aplicada no código
RN01	Uma reserva não pode se sobrepor a outra reserva ativa na mesma área e data	<code>ReservaService::criar</code>
RN02	A reserva deve respeitar o horário de funcionamento da área	<code>AreaComum::validarPeriodo</code> , chamado por <code>validarReserva</code>
RN03	O número de convidados não pode passar da capacidade da área	<code>AreaComum::validarPeriodo</code> e <code>calcularTaxa</code>
RN04	O cancelamento exige 24 horas de antecedência; a reserva cancelada libera o horário	<code>ReservaService::cancelar</code>
RN05	Só funcionário com cargo de porteiro registra entrada de visitante	<code>VisitaService::registrarEntrada</code>
RN06	A saída não pode ser anterior à entrada	<code>Visita::setSaida</code> e <code>VisitaService::registrarSaida</code>
RN07	O CPF deve ser válido (11 dígitos e dígitos verificadores) e único entre todas as pessoas	<code>Pessoa::cpfValido</code> e os serviços de cadastro
RN08	Apartamento com moradores e morador com reservas não podem ser removidos	<code>ApartamentoService::remover</code> e <code>MoradorService::remover</code> , apoiados nas chaves estrangeiras do banco

Cada área comum aplica suas próprias regras de duração e taxa. A capacidade e o horário de funcionamento são informados no cadastro da área.

Tabela 5 – Regras de duração e taxa de cada área comum

Área	Duração máxima	Regra de convidados	Taxa (<code>calcularTaxa</code>)
SalaoFestas	8 horas	Até a capacidade da área	Taxa base da área mais R\$ 3,00 por convidado
Piscina	2 horas	No máximo 4 convidados	Gratuita
Churrasqueira	4 horas	Até a capacidade da área	R\$ 40,00 fixos

3. Arquitetura em camadas e estrutura do código

Dividimos o sistema em camadas para que cada integrante pudesse desenvolver e testar seu módulo sem depender dos outros. A regra principal é que uma camada só conversa com a

camada logo abaixo dela. A interface nunca escreve SQL, e o repositório nunca decide regra de negócio.

Tabela 6 – Camadas da arquitetura

Camada	Responsabilidade	Exemplos
Interface	Menus de terminal e telas Qt; recebe os dados do usuário e mostra os resultados e os erros	MenuMoradores, MenuReservas, TelaMoradores, TelaReservas, TelaVisitas
Serviço	Regras de negócio e funções do usuário final	ReservaService, VisitaService, MoradorService
Repositório	CRUD no banco, isolando o SQL	IREpositorio<T>, RepositorioPessoa, RepositorioReserva
Infraestrutura	Conexão com o banco, exceções e fábricas	Database, ErroCondominio, FabricaPessoa, FabricaAreaComum
Modelo	Entidades usadas por todas as camadas	Pessoa, Morador, AreaComum, Reserva

O caminho de uma operação é sempre o mesmo: interface → serviço → repositório → SQLite. Por exemplo, ao criar uma reserva, a tela chama `ReservaService::criar`, que valida as regras e pede ao repositório para gravar. Se alguma regra falha, o serviço lança uma exceção e nada é gravado.

O CMake monta o núcleo como uma biblioteca chamada `condominio_core`, que não usa Qt. Em cima dela são criados dois executáveis: `condominio_terminal` e `condominio_gui`. Como os dois usam exatamente os mesmos serviços, o comportamento do sistema é o mesmo nas duas formas de uso. Se o Qt 6 não for encontrado na máquina, o CMake compila apenas o núcleo, o terminal e os testes.

O SQLite está dentro do repositório (`third_party/sqlite`), então quem clona o projeto não precisa instalar o banco.

—	CMakeLists.txt
—	README.md
—	docs/ relatório, diagramas, guias e roteiro do vídeo
—	sql/ schema.sql e seed.sql
—	include/ cabeçalhos (.h)
—	modelo/ Pessoa, Morador, Visitante, Funcionario, Apartamento, Visita, AreaComum, SalaoFestas, Piscina, Churrasqueira, Reserva
—	infra/ Database, ErroCondominio, fábricas
—	repositorio/ IRepositorio e um repositório por entidade
—	servico/ um serviço por módulo (ReservaService, etc.)
—	interface/ menus de terminal e telas Qt
—	src/ implementações (.cpp), mesmas pastas
—	terminal/ main_terminal.cpp
—	main.cpp ponto de entrada da interface Qt
—	tests/ testes de integração (CTest)
—	third_party/sqlite/ código do SQLite

O banco `condominio.db` é criado automaticamente na primeira execução, a partir do arquivo `sql/schema.sql`. O arquivo `sql/seed.sql` tem dados fictícios para demonstração e não é aplicado sozinho.

4. Modelo de classes e diagrama UML

As Figuras 1 a 5 mostram as classes do modelo, as exceções e os enums, exatamente como estão no código. Para que o texto fique legível na página, o diagrama foi dividido em partes. Uma classe que aparece em mais de uma figura (como `Morador`) é detalhada na primeira e aparece só com o nome nas outras. O diagrama completo, em uma única imagem, está em `docs/diagrama-classes.png` e `docs/diagrama-classes.pdf`.

Para ler as figuras: a linha contínua com triângulo vazado indica herança; o losango vazado indica agregação; a seta contínua aberta liga uma classe a um enum; e a seta tracejada indica uma dependência por id guardado (a classe guarda o número de identificação do outro objeto, e não um ponteiro).

Figura 1 – Hierarquia de pessoas

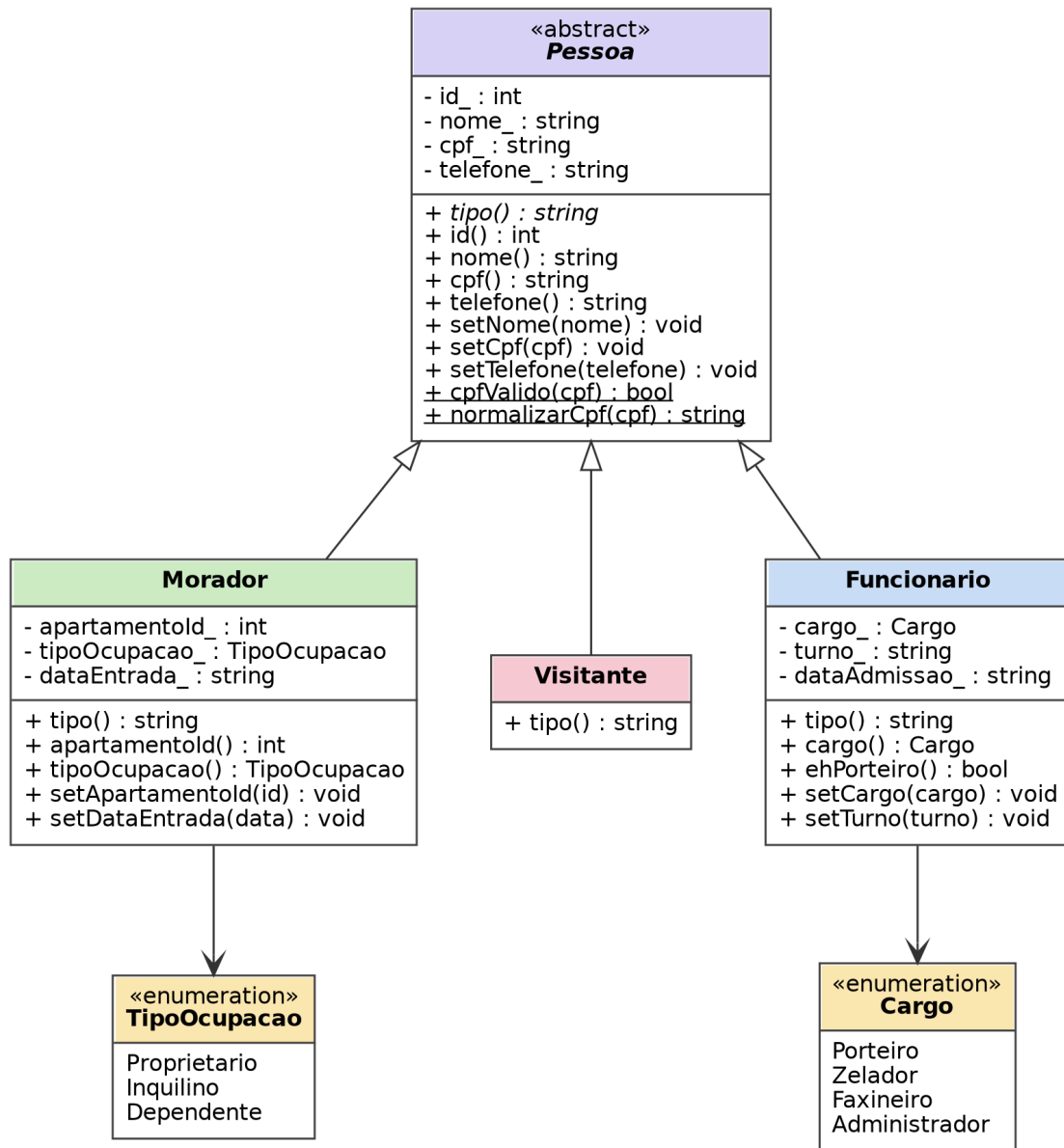


Figura 2 – Apartamento e visitas

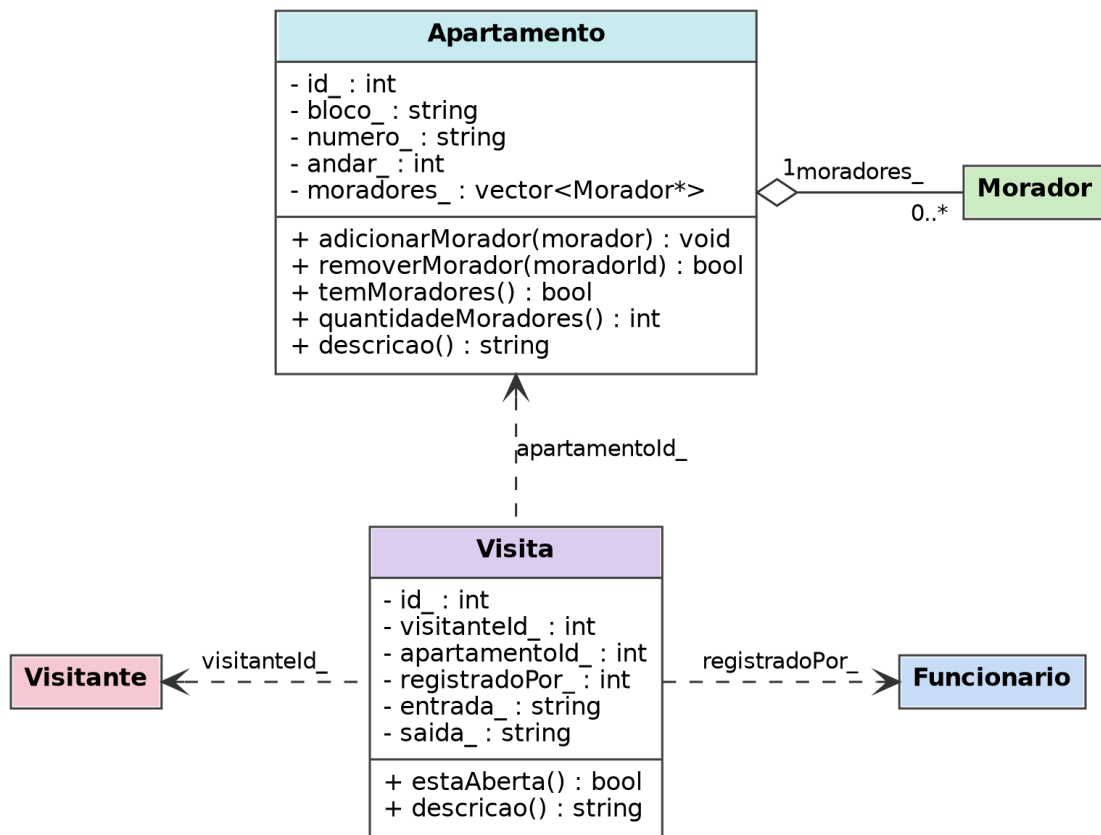


Figura 3 – Reservas

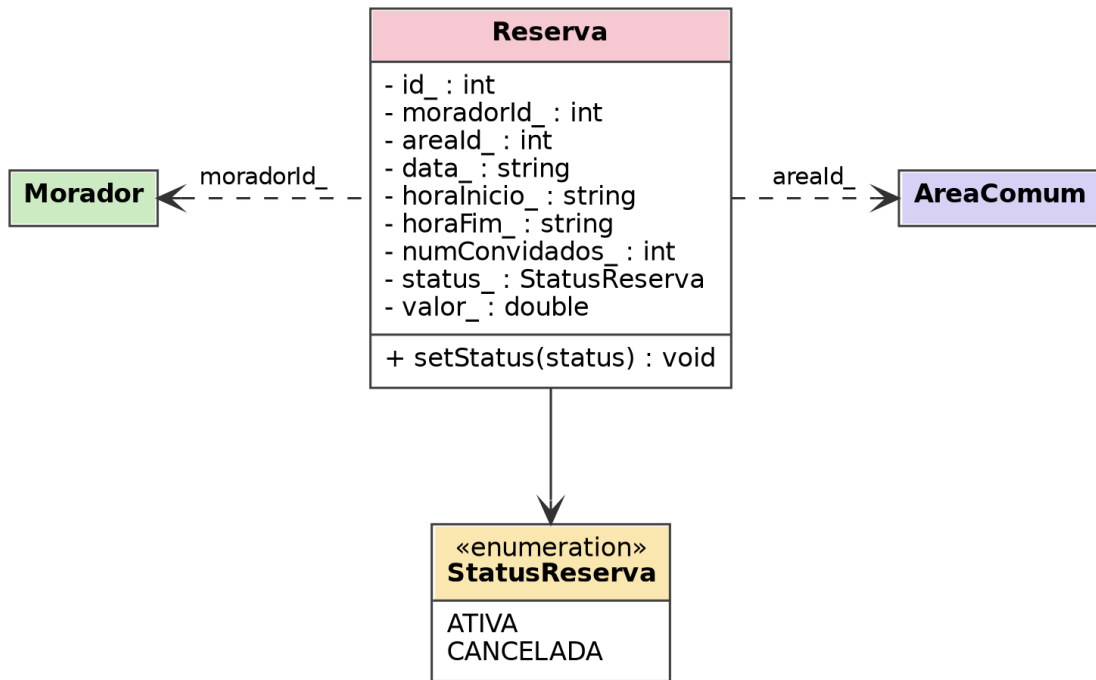


Figura 4 – Hierarquia de áreas comuns

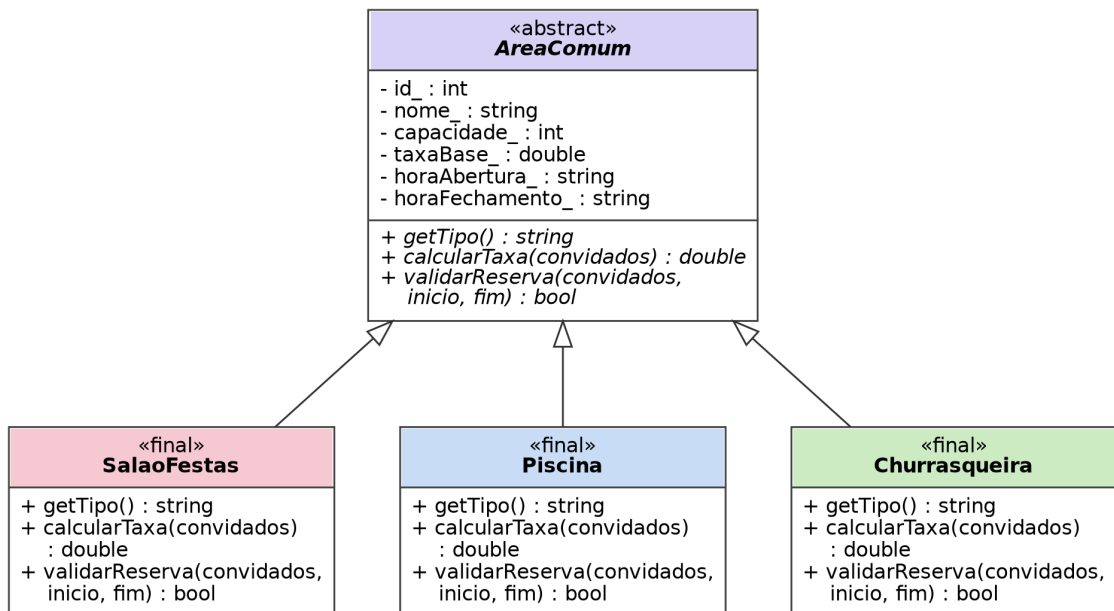
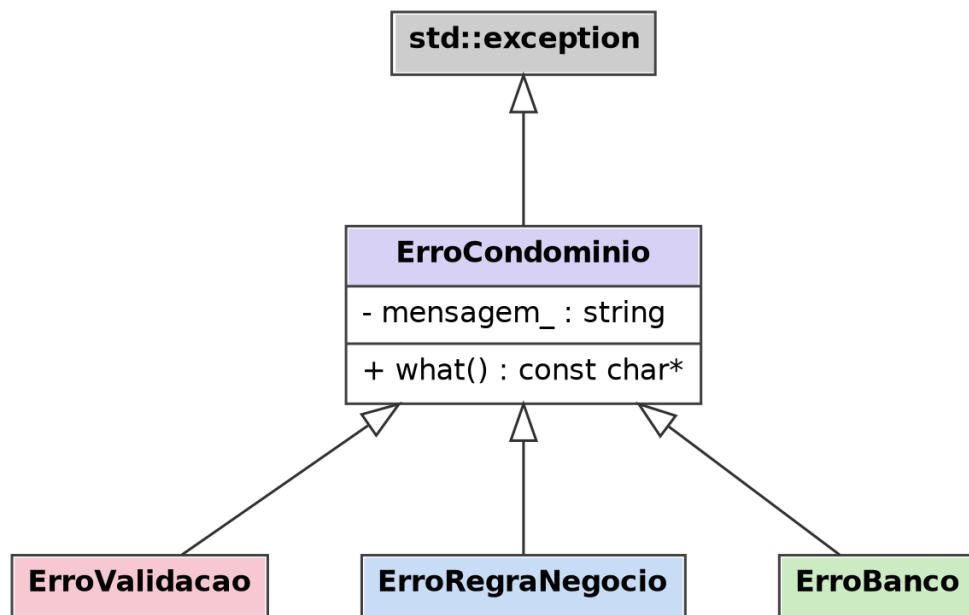


Figura 5 – Hierarquia de exceções



O modelo tem três árvores de herança:

- **Pessoa** (classe abstrata), com as subclasses **Morador**, **Visitante** e **Funcionario**.
- **AreaComum** (classe abstrata), com as subclasses **SalaoFestas**, **Piscina** e **Churrasqueira**, todas marcadas como **final**.
- **ErroCondominio**, que herda de **std::exception**, com as subclasses **ErroValidacao**, **ErroRegraNegocio** e **ErroBanco**.

As classes **Apartamento**, **Reserva** e **Visita** não usam herança, porque não existem variações delas que justifiquem subclasses. Elas se ligam às outras por associação:

- **Apartamento** guarda um **vector<Morador*>** com ponteiros que não são donos dos objetos (agregação). Por isso o losango fica do lado do apartamento, e a ligação é com **Morador**, não com **Pessoa**.
- **Reserva** guarda apenas os ids do morador e da área comum. Esses ids são os mesmos que ficam no banco, o que facilita gravar e ler a reserva.
- **Visita** guarda os ids do visitante, do apartamento visitado e do funcionário (porteiro) que registrou a entrada. Cada visita é um evento separado, para que o mesmo visitante possa ter várias visitas no histórico.

As setas tracejadas do diagrama representam essas ligações por id.

Três enums (**enum class**) representam conjuntos fechados de valores: **TipoOcupacao** (Proprietario, Inquilino, Dependente), **Cargo** (Porteiro, Zelador, Faxineiro, Administrador) e **StatusReserva** (ATIVA, CANCELADA).

Uma decisão de projeto importante foi tratar o cargo do funcionário como um atributo (o enum `Cargo`) e não criar subclasses como `Porteiro` e `Zelador`. A única diferença de comportamento entre os cargos é a regra de que só o porteiro registra visitas, e isso é resolvido com o método `ehPorteiro()`. Assim, mantivemos a hierarquia simples e mais fácil de explicar.

5. Conceitos de POO usados e como foram usados

A tabela resume onde cada conceito aparece. Nas subseções seguintes, cada um é explicado com um trecho curto do código.

Tabela 7 – Conceitos de POO e onde aparecem

Conceito	Onde aparece	Como foi usado
Classes e objetos	Todas as entidades do modelo	Cada entidade do condomínio é uma classe, com atributos e métodos próprios
Encapsulamento e modificadores de acesso	<code>Pessoa</code> , <code>AreaComum</code> , <code>Apartamento</code>	Atributos <code>private</code> , métodos auxiliares <code>protected</code> e interface <code>public</code> , com setters que validam
Herança	<code>Pessoa</code> , <code>AreaComum</code> , <code>ErroCondominio</code>	Subclasses reaproveitam atributos e regras da classe-base
Classe abstrata e método virtual puro	<code>Pessoa</code> , <code>AreaComum</code> , <code>IRepositorio<T></code>	A classe-base define um contrato que as subclasses são obrigadas a cumprir
Polimorfismo	<code>AreaComum</code> e suas subclasses	Cada área calcula a taxa e valida a reserva à sua maneira, com <code>virtual</code> e <code>override</code>
Ponteiros e referências	Repositórios, serviços e <code>Apartamento</code>	<code>unique_ptr</code> , referências <code>const&</code> e ponteiros que não são donos do objeto
Templates	<code>IRepositorio<T></code> e <code>FabricaPorTipo<Base></code>	Um único contrato ou fábrica reaproveitado para várias entidades
Tratamento de exceções	<code>ErroCondominio</code> e subclasses	Três tipos de erro, tratados pela interface

5.1 Classes, objetos e encapsulamento

Os atributos das classes ficam `private` e terminam com `_`, como `nome_` e `capacidade_`. Para ler ou alterar um valor, a classe oferece métodos públicos. Os setters validam o dado antes de guardá-lo, então um objeto nunca fica com um valor inválido, como um CPF com dígitos errados. Em `Pessoa`, por exemplo, os atributos são privados e os métodos de apoio são `protected`, ou seja, só as subclasses podem usá-los:

```

class Pessoa {
private:
    int id_ = 0;
    string nome_;
    string cpf_;
    string telefone_;
protected:
    static string textoObrigatorio(const string& valor, const string& campo);
    static string dataIsoValida(const string& data, const string& campo);
public:
    Pessoa(string nome, string cpf, string telefone, int id = 0);
    virtual string tipo() const = 0;
    void setNome(const string& nome);
    void setCpf(const string& cpf);
    // ...
};

```

O método `setNome` usa `textoObrigatorio` e lança `ErroValidacao` se o nome vier vazio. `Morador` e `Funcionario` usam `dataIsoValida` para conferir datas, sem precisar repetir o código.

5.2 Herança

A herança evita repetir código. `Pessoa` guarda `id_`, `nome_`, `cpf_` e `telefone_` e valida o CPF uma única vez. `Morador` acrescenta apartamento, tipo de ocupação e data de entrada. `Funcionario` acrescenta cargo, turno e data de admissão. `Visitante` não acrescenta nada, porque só precisa ser distinguível dos outros tipos. O construtor da subclasse chama o da classe-base:

```

Morador::Morador(string nome, string cpf, string telefone, int apartamentoId,
                  TipoOcupacao tipoOcupacao, string dataEntrada, int id)
: Pessoa(nome, cpf, telefone, id), tipoOcupacao_(tipoOcupacao) {
    setApartamentoId(apartamentoId);
    setDataEntrada(dataEntrada);
}

```

O trecho `: Pessoa(nome, cpf, telefone, id)` é onde a subclasse passa os dados comuns para a classe-base.

5.3 Classes abstratas e métodos virtuais puros

`Pessoa` e `AreaComum` são abstratas: têm pelo menos um método virtual puro (`= 0`), então não é possível criar um objeto direto delas. Só existem objetos de `Morador`, `Visitante`, `Funcionario`, `SalaoFestas`, `Piscina` e `Churrasqueira`. Em `AreaComum`, as três regras que variam de uma área para outra são virtuais puras:

```

class AreaComum {
public:
    virtual ~AreaComum() = default;
    virtual std::string getTipo() const = 0;
    virtual double calcularTaxa(int convidados) const = 0;
    virtual bool validarReserva(int convidados, const std::string& inicio,
                                const std::string& fim) const = 0;
protected:
    bool validarPeriodo(int convidados, const std::string& inicio,
                        const std::string& fim, int duracaoMaximaMinutos) const;
    // ...
};

```

O destrutor é **virtual** para que o objeto seja destruído corretamente quando for apagado por um ponteiro para a classe-base. O método **validarPeriodo** é **protected** e fica na classe-base porque as três subclasses usam a mesma verificação de horário e capacidade, mudando apenas a duração máxima. **IRepositorio<T>** também é uma interface abstrata: só declara os cinco métodos do CRUD e não tem implementação.

5.4 Polimorfismo

Polimorfismo é chamar o mesmo método em objetos diferentes e cada um responder à sua maneira. Em **AreaComum**, cada subclasse sobrescreve (**override**) o cálculo da taxa:

```

// SalaoFestas: taxa base mais R$ 3,00 por convidado
double SalaoFestas::calcularTaxa(int convidados) const {
    if (convidados < 0 || convidados > getCapacidade()) {
        throw ErroValidacao("Numero de convidados invalido");
    }
    return getTaxaBase() + 3.0 * convidados;
}

// Churrasqueira: taxa fixa
double Churrasqueira::calcularTaxa(int convidados) const {
    // ... mesma validação
    return 40.0;
}

```

A vantagem aparece no **ReservaService::criar**. Ele busca a área no repositório e a recebe como um **unique_ptr<AreaComum>**, sem saber se é salão, piscina ou churrasqueira. Mesmo assim, chama os mesmos métodos, e cada objeto aplica a sua regra:

```

auto area = repoArea->buscarPorId(areaId);
if (!area->validarReserva(convidados, horaInicio, horaFim)) {
    throw ErroRegraNegocio("Capacidade, duracao ou horario fora das regras da area");
}
// ...
Reserva nova(0, moradorId, areaId, data, horaInicio, horaFim,
             convidados, StatusReserva::ATIVA, area->calcularTaxa(convidados));

```

Se um dia o condomínio ganhar uma quadra, basta criar uma nova subclasse. O **ReservaService** não precisa mudar.

Em `Pessoa`, o método virtual puro `tipo()` devolve "morador", "visitante" ou "funcionario". Ele é usado para gravar o tipo da pessoa no banco. Os repositórios e serviços trabalham com `unique_ptr<Pessoa>`, e, quando precisam de um dado específico de uma subclasse (por exemplo, o cargo de um `Funcionario`), usam `dynamic_cast`:

```
unique_ptr<Pessoa> pessoa = repoPessoa_.buscarPorId(porteiroId);
Funcionario* porteiro = dynamic_cast<Funcionario*>(pessoa.get());
if (porteiro == nullptr || !porteiro->ehPorteiro()) {
    throw ErroRegraNegocio("so um funcionario com cargo de porteiro registra visitas");
}
```

O `dynamic_cast` devolve `nullptr` se a pessoa não for um funcionário, o que permite recusar a operação sem travar o programa. Reconhecemos que esse uso é mais uma conferência de tipo do que polimorfismo puro, e que o exemplo mais claro do conceito no projeto é a hierarquia de `AreaComum`.

5.5 Ponteiros e referências

Evitamos `new` e `delete` manuais nas classes do núcleo. Em vez disso, usamos ponteiros inteligentes e referências:

- `std::unique_ptr` em coleções polimórficas. Quem recebe o resultado de `buscarPorId` ou `listar` passa a ser dono do objeto, e a memória é liberada sozinha. Isso permite devolver um `Morador` pelo tipo `Pessoa` sem perder as partes da subclasse.
- Referências `const&` nos parâmetros (por exemplo, `const string& nome`), para não copiar o objeto à toa e deixar claro que o método não o altera.
- Referências para os repositórios nos serviços, como `IREpositorio<Pessoa>& repoPessoa`, para o serviço usar o repositório sem ser dono dele.
- Ponteiros que não são donos em `Apartamento`, que guarda um `vector<Morador*>`. O comentário no código lembra que o apartamento nunca dá `delete` nesses objetos.

O `new` aparece apenas na interface Qt, ao criar janelas e botões. Nesse caso, o próprio Qt apaga os objetos quando a janela-mãe é destruída.

5.6 Templates

`IREpositorio<T>` é um modelo de classe (template): o mesmo contrato de CRUD serve para `Apartamento`, `Pessoa`, `Visita` e as demais entidades.

```
template <typename T>
class IRepositorio {
public:
    virtual ~IRepositorio() = default;
    virtual int inserir(const T& entidade) = 0;
    virtual std::unique_ptr<T> buscarPorId(int id) = 0;
    virtual std::vector<std::unique_ptr<T>> listar() = 0;
    virtual bool atualizar(const T& entidade) = 0;
    virtual bool remover(int id) = 0;
};
```

O `FabricaPorTipo<Base>`, usado pelas fábricas, também é um template.

5.7 Exceções

Os erros do sistema são classes que herdam de `ErroCondominio`, que por sua vez herda de `std::exception`. Cada tipo indica a origem do problema: `ErroValidacao` (dado inválido, como CPF errado), `ErroRegraNegocio` (regra quebrada, como conflito de horário) e `ErroBanco` (falha no SQLite). As camadas de baixo lançam o erro e a interface decide como mostrá-lo: o terminal imprime a mensagem, e o Qt abre uma janela de aviso.

6. Design patterns

Design patterns são soluções conhecidas para problemas que se repetem no desenvolvimento de software. O enunciado cita o uso de design patterns como um ponto extra (bônus). Usamos três, que resolvem problemas reais do nosso sistema: Singleton, Repository e Factory.

Tabela 8 – Design patterns utilizados

Padrão	Problema que resolve	Onde está
Singleton	Garantir uma única conexão com o banco em todo o sistema	<code>Database</code>
Repository	Separar o SQL do restante do sistema	<code>IRepositorio<T></code> e um repositório por entidade
Factory	Criar a subclasse certa a partir do tipo salvo no banco	<code>FabricaPessoa</code> e <code>FabricaAreaComum</code>

6.1 Singleton e RAII

O sistema precisa de uma única conexão SQLite. Se cada classe abrisse a sua, os dados poderiam ficar inconsistentes. Por isso `Database` é um Singleton: o construtor é `private`, a cópia e a movimentação são apagadas (= `delete`) e o acesso é feito por um método estático.

```
Database& Database::instancia(const std::string& caminhoBanco,
                             const std::string& caminhoSchema) {
    static Database unicaInstancia(caminhoBanco, caminhoSchema);
    return unicaInstancia;
}
```

A primeira chamada abre o banco, ativa as chaves estrangeiras (`PRAGMA foreign_keys = ON`) e cria as tabelas, caso o banco ainda esteja vazio. As chamadas seguintes reaproveitam a mesma conexão. O destrutor fecha o banco.

Essa classe também aplica RAII (*Resource Acquisition Is Initialization*): o recurso é aberto no construtor e fechado no destrutor, então a conexão é liberada mesmo se acontecer uma exceção. Reconhecemos uma limitação: como a conexão é global, a solução serve bem para um aplicativo local de um usuário, mas precisaria ser revista para vários bancos ou acessos simultâneos.

6.2 Repository

O padrão Repository esconde do restante do sistema como os dados são gravados. Os serviços conversam com `IRepositorio<T>` e não sabem que existe SQLite por trás. Cada entidade tem uma implementação (`RepositorioApartamento`, `RepositorioPessoa`, `RepositorioVisita`, `RepositorioAreaComum` e `RepositorioReserva`), e todo o SQL fica dentro dessas classes.

Isso trouxe duas vantagens. A primeira é que os serviços podem ser testados com um banco SQLite temporário em memória. A segunda é que, se um dia o banco mudar (por exemplo, para PostgreSQL), só as classes de repositório precisam ser reescritas.

O `RepositorioPessoa` mostra um cuidado importante: uma pessoa é gravada em duas tabelas (`pessoa` e `morador`, ou `pessoa` e `funcionario`). Para que as duas gravações aconteçam juntas ou nenhuma aconteça, usamos uma classe `Transacao`. Ela abre uma transação no construtor e, se não for confirmada, desfaz tudo no destrutor.

```
class Transacao {
public:
    Transacao() { Database::instancia().executar("BEGIN"); }
    ~Transacao() {
        if (!confirmada_) {
            sqlite3_exec(conexao(), "ROLLBACK", nullptr, nullptr, nullptr);
        }
    }
    void confirmar() {
        Database::instancia().executar("COMMIT");
        confirmada_ = true;
    }
    // ...
};
```

6.3 Factory

O banco guarda apenas um texto com o tipo do objeto (como `"morador"` ou `"Piscina"`), mas o sistema precisa devolver um objeto C++ da subclasse correta. A fábrica resolve isso. A classe `FabricaPorTipo<Base>` guarda, para cada texto, uma função que sabe construir a subclasse. As

classes `FabricaPessoa` e `FabricaAreaComum` apenas definem os tipos aceitos. Um tipo desconhecido é recusado com uma exceção.

No `RepositorioPessoa`, ao ler uma linha do banco, o repositório registra como construir cada tipo e pede à fábrica o objeto do tipo da linha:

```
FabricaPessoa fabrica;
fabrica.registrarMorador([&]() -> unique_ptr<Pessoa> {
    return make_unique<Morador>(nome, cpf, telefone, apartamentoId,
                                textoParaOcupacao(ocupacao), dataEntrada, id);
});
// ... registra Funcionario e Visitante da mesma forma
return fabrica.criar(tipo);
```

O resultado é um `unique_ptr<Pessoa>` que contém, de fato, um `Morador`, um `Funcionario` ou um `Visitante`. Assim, o polimorfismo continua funcionando depois que o objeto sai do banco, e a decisão de qual subclasse criar fica concentrada em um só lugar, sem se espalhar pelas telas.

O padrão Observer, previsto no planejamento para atualizar as telas automaticamente com os signals e slots do Qt, não foi implementado (ver a seção 11). Hoje as telas se atualizam quando o usuário muda de aba.

7. Modelo de dados e diagrama ER

O banco tem sete tabelas, criadas pelo arquivo `sql/schema.sql`. O diagrama entidade-relacionamento (ER) da Figura 6 mostra as tabelas, as chaves primárias (PK), as chaves estrangeiras (FK) e os campos únicos (UQ). Ele também está em `docs/diagrama-er.png`.

Figura 6 – Diagrama entidade-relacionamento do banco de dados

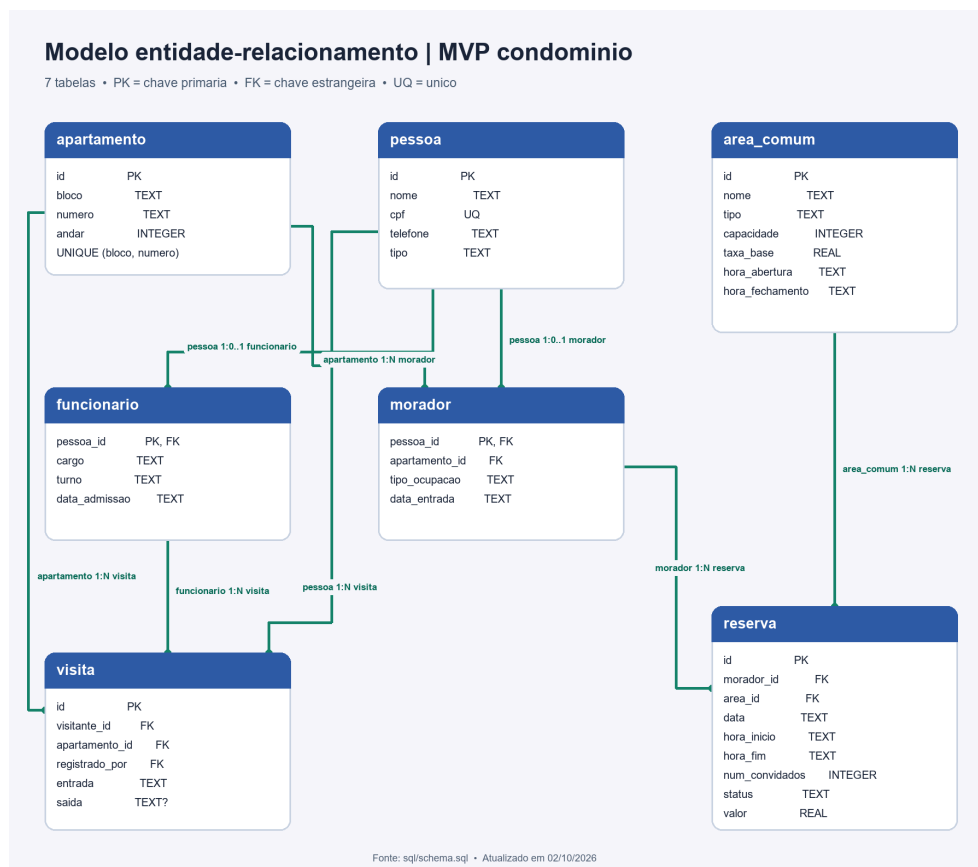


Tabela 9 – Tabelas do banco de dados

Tabela	Campos principais	Restrições
apartamento	id, bloco, numero, andar	Combinação (bloco, numero) única
pessoa	id, nome, cpf, telefone, tipo	CPF único com 11 dígitos; tipo em morador, visitante ou funcionario
morador	pessoa_id, apartamento_id, tipo_ocupacao, data_entrada	pessoa_id é PK e FK; apartamento é obrigatório
funcionario	pessoa_id, cargo, turno, data_admissao	pessoa_id é PK e FK; cargo e turno obrigatórios
area_comum	id, nome, tipo, capacidade, taxa_base, hora_abertura, hora_fechamento	Capacidade maior que zero; fechamento depois da abertura
reserva	id, morador_id, area_id, data, hora_inicio, hora_fim, num_convitados, status, valor	hora_fim maior que hora_inicio; status ativa ou cancelada
visita	id, visitante_id, apartamento_id, registrado_por, entrada, saida	A saída, quando existe, não pode ser anterior à entrada

Algumas decisões de modelagem merecem explicação:

- **Herança no banco.** A tabela **pessoa** guarda os dados comuns e a coluna **tipo** indica de que tipo a pessoa é. As tabelas **morador** e **funcionario** guardam os dados específicos e usam **pessoa_id** como chave primária e estrangeira ao mesmo tempo, então cada pessoa

tem no máximo uma linha de cada subtipo. O visitante não precisa de dados extras, por isso não tem tabela própria.

- **Visita separada de visitante.** Cada visita é uma linha em **visita**. Assim, a mesma pessoa pode ter várias visitas, e o histórico é preservado. Uma visita sem saída registrada (**saida** vazia) está em aberto.
- **Datas e horas como texto.** Datas ficam no formato **AAAA-MM-DD** e horas no formato **HH:MM**, com verificação no próprio banco. Nesse formato, comparar os textos funciona como comparar as datas.
- **Chaves estrangeiras com ON DELETE RESTRICT.** O banco recusa apagar um apartamento que tem moradores, ou um morador que tem reservas. Os serviços aproveitam esse erro para mostrar uma mensagem de regra de negócio (RN08).

As chaves estrangeiras só funcionam se forem ativadas em cada conexão. Por isso a classe **Database** executa **PRAGMA foreign_keys = ON** ao abrir o banco.

Um detalhe da modelagem: o campo de ocupação do morador e o de cargo do funcionário são gravados como texto, e os repositórios convertem esse texto para os enums **TipoOcupacao** e **Cargo** ao montar os objetos.

8. Interface gráfica e terminal

O sistema pode ser usado de duas formas, e as duas chamam os mesmos serviços do núcleo. Por isso, nenhuma regra de negócio e nenhum SQL ficam na interface.

8.1 Interface gráfica (Qt 6)

O executável **condominio_gui** abre uma janela principal com três abas. Cada aba reúne o CRUD de um conjunto de entidades:

Tabela 10 – Abas da interface gráfica

Aba	O que permite fazer
Moradores e Aptos	Cadastrar, editar e remover apartamentos e moradores. Ao clicar numa linha da tabela, o formulário é preenchido para facilitar a edição
Áreas e Reservas	Cadastrar áreas comuns, criar reservas escolhendo morador, área, data e horário, consultar e cancelar reservas
Portaria e Visitas	Cadastrar funcionários e visitantes, registrar entrada (escolhendo visitante, apartamento e porteiro) e saída, e consultar o histórico por apartamento

Quando uma regra de negócio é quebrada (por exemplo, horário em conflito, CPF inválido ou tentativa de remover um apartamento ocupado), a tela captura a exceção e mostra a mensagem em uma janela de aviso (**QMessageBox**). Os dados são atualizados quando o usuário muda de aba.

A aparência da interface segue um tema em tons de azul, definido em uma folha de estilo do Qt (QSS, parecida com CSS) e aplicado no `main.cpp` sobre o estilo base `Fusion`, que fica igual em qualquer sistema operacional. Os botões de remover e de cancelar aparecem em vermelho para destacar as ações que apagam dados.

A interface foi feita com Qt Widgets. As classes das telas (`TelaMoradores`, `TelaReservas` e `TelaVisitas`) herdam de `QWidget`, o que é mais um uso de herança no projeto. A janela principal organiza as telas em abas com um `QTabWidget`.

O funcionamento das telas é demonstrado no vídeo do projeto, cujo link está na seção 1.

8.2 Terminal

O executável `condominio_terminal` abre um menu em texto com duas opções: apartamentos, moradores e funcionários; e áreas comuns e reservas. Cada submenu oferece cadastrar, listar, editar e remover, e as mensagens de erro dos serviços são impressas na tela. O terminal foi o primeiro meio de uso do sistema: desenvolvemos e testamos o núcleo por ele antes de ligar a interface gráfica.

O registro de visitas ainda não tem menu no terminal e só pode ser feito pela interface gráfica.

8.3 Como executar

```
cmake -S . -B build
cmake --build build
cd build
./condominio_terminal    # versão de terminal
./condominio_gui         # interface gráfica (precisa do Qt 6)
```

Na primeira execução, o banco `condominio.db` é criado na pasta de onde o programa foi iniciado. Se o CMake não encontrar o Qt, é preciso informar o caminho da instalação com `-DCMAKE_PREFIX_PATH=<caminho-do-Qt>`.

9. Testes e resultados

Os testes automatizados ficam na pasta `tests/` e são executados pelo CTest, que faz parte do CMake. Cada teste é um programa pequeno que usa os serviços e repositórios reais sobre um banco SQLite temporário em memória, criado a partir de `sql/schema.sql`. Assim, os testes não alteram o banco usado pela aplicação e podem ser repetidos quantas vezes for preciso. Se alguma verificação falha, o programa termina com erro e o CTest mostra qual teste falhou.

Para executar:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure
```

9.1 Cenários de integração (D02)

O teste `cenarios_integracao_d02` reúne dez cenários que percorrem os fluxos principais do sistema, da camada de serviço até o banco. O relógio do teste de reservas é fixado em uma data conhecida para que o resultado não dependa do dia da execução.

Tabela 11 – Cenários de teste de integração (D02)

Cenário	Fluxo verificado	Resultado
D02-01	Inicializar um banco vazio e encontrar as sete tabelas do MVP	Passou
D02-02	Cadastrar e editar um apartamento e ler a alteração gravada	Passou
D02-03	Cadastrar um morador vinculado a um apartamento e listá-lo pelo apartamento	Passou
D02-04	Cadastrar um funcionário e recusar um CPF já usado por um morador	Passou
D02-05	Impedir a remoção de um apartamento que tem morador, preservando o registro	Passou
D02-06	Recusar entrada de visita registrada por um zelador e aceitar a de um porteiro	Passou
D02-07	Registrar a saída e conferir visitas abertas e histórico por apartamento	Passou
D02-08	Cadastrar e editar uma área comum e reconstruir o subtipo <code>Piscina</code> a partir do banco	Passou
D02-09	Criar reserva; recusar conflito de horário, horário fechado e excesso de capacidade; conferir o valor	Passou
D02-10	Cancelar com antecedência e liberar o horário; recusar cancelamento com menos de 24 horas	Passou

9.2 Demais testes automatizados

Além dos cenários, cada módulo tem testes próprios, escritos conforme os módulos foram integrados.

Tabela 12 – Demais testes automatizados

Teste	Parte do sistema coberta
<code>terminal_inicia</code>	O terminal inicia e o banco é criado (<code>--smoke</code>)
<code>database_i02</code>	Conexão SQLite (Singleton e RAII)
<code>irepositorio_i04</code>	Contrato <code>IRepositorio<T></code>
<code>fabricas_i06</code> e <code>fabrica_pessoa_real_i06</code>	Fábricas e criação da subclasse correta
<code>integracao_pessoas_i06</code> e <code>servicos_pessoas_p05</code>	Repositório e serviços de apartamentos, moradores e funcionários

Teste	Parte do sistema coberta
<code>integracao_visitas</code>	Visitas gravadas e lidas do banco
<code>modelos_areas_reservas</code> , <code>repositorios_areas_reservas</code> e <code>servicos_areas_reservas</code>	Áreas comuns, reservas e suas regras
<code>cenarios_integracao_d02</code>	Os dez cenários da seção 9.1
<code>gui_fluxos</code>	Fluxos das telas Qt, com o plugin <code>offscreen</code> (só é compilado quando o Qt é encontrado)

9.3 Resultados

- Em 02/10/2026, em Windows com MSVC 19.51, a compilação do núcleo passou e o CTest registrou 11 de 11 testes aprovados. O Qt não estava instalado nessa máquina, então esse resultado não inclui a interface gráfica.
- Na versão final da `main`, em Linux (Ubuntu, GCC 13.3), sem Qt, os 12 testes passaram, e o núcleo compilou sem warnings com `-Wall -Wextra -Wpedantic`. O teste `gui_fluxos` só é registrado quando o Qt é encontrado, por isso não está entre esses 12.

Os testes automáticos cobrem o núcleo do sistema: modelo, repositórios, serviços, regras de negócio e fluxos de integração. O funcionamento da interface gráfica é demonstrado no vídeo do projeto.

10. Divisão de trabalho

A divisão de trabalho seguiu o Plano de Execução do grupo, de 21/09/2026. O sistema foi dividido em módulos, e cada integrante ficou como responsável por um deles e por escrever e narrar, na documentação e no vídeo, a parte que desenvolveu. Cada integrante também assumiu um papel da metodologia ágil adotada (Scrum leve, com quadro Kanban). Nas tarefas maiores, o plano previa um apoio, que trabalha junto com o responsável, e todo o código deveria entrar na `main` por Pull Request revisado por outra pessoa do grupo.

Tabela 13 – Divisão de trabalho da equipe

Integrante	Papel	Responsabilidade técnica	Documentação e vídeo
Vivian Azevedo	Scrum Master	Pessoas (<code>Pessoa</code> , <code>Morador</code> , <code>Visitante</code> e <code>Funcionario</code>), apartamentos, funcionários e o backend de visitas; exceções (<code>ErroCondominio</code> e subclasses); repositórios e serviços desses módulos	Diagrama de classes; seção de pessoas, herança e polimorfismo; consolidação do relatório; parte do vídeo sobre pessoas e visitas; condução das cerimônias
Clécio Muniz	Tech Lead	Banco de dados (<code>schema.sql</code> , <code>seed.sql</code> e <code>Database</code>), contrato <code>IRepositorio<T></code> , fábricas, CMake e testes de integração	Diagrama ER; seção de arquitetura e design patterns; roteiro, edição e publicação do vídeo; criação das tags de versão

Integrante	Papel	Responsabilidade técnica	Documentação e vídeo
Sâmia Freitas	Product Owner	Interface Qt (janela principal e telas), wireframes, menu principal do terminal e serviço de visitas	Seção da interface gráfica; README final; parte do vídeo de abertura e interface; aceite das histórias e priorização
Beatriz Luna	Dev líder de Reservas	Áreas comuns (<i>AreaComum</i> e subclasses), reservas e suas regras (conflito de horário, capacidade, horário de funcionamento e cancelamento), repositórios e serviços desse módulo, menu de terminal de áreas e reservas	Seção de áreas comuns, polimorfismo e regras de reserva; página github.io; parte do vídeo sobre áreas e reservas

Algumas tarefas foram feitas em dupla. O plano previa, por exemplo, que Clécio apoiasse Vivian e Beatriz nos repositórios e serviços, e que Vivian, Clécio e Beatriz apoiassem Sâmia nas telas da interface. A consolidação final do relatório ficou com Vivian.

11. Dificuldades, aprendizados e trabalhos futuros

11.1 Dificuldades

- **Integrar módulos feitos em paralelo.** Cada integrante trabalhou em uma branch e os módulos entraram na *main* por Pull Requests. Na integração, foi preciso ajustar o CMake, reescrever o *VisitaService* e ligar áreas e reservas ao SQLite. Definir os contratos no início (*IRepositorio<T>* e os cabeçalhos dos serviços) reduziu o retrabalho, mas não eliminou esses ajustes.
- **Configurar o Qt 6.** A interface gráfica depende do Qt 6, que exige configurar a versão e o caminho de instalação no CMake. Em uma das máquinas ele não foi encontrado, e por isso o CMake foi configurado para compilar só o núcleo, o terminal e os testes quando o Qt não está disponível. Isso permitiu continuar testando mesmo sem a interface gráfica.
- **Equilibrar polimorfismo e checagem de tipo.** Em alguns serviços usamos *dynamic_cast* para saber se uma *Pessoa* é um *Morador*, um *Funcionario* ou um *Visitante*. Funciona, mas é mais uma conferência de tipo do que polimorfismo. O exemplo mais claro do conceito é a hierarquia de *AreaComum*.
- **Ids em vez de ponteiros.** *Reserva* e *Visita* guardam apenas ids. Isso simplifica a gravação no banco, mas obriga o serviço a buscar os objetos relacionados quando precisa deles.

11.2 Aprendizados

- Dividir o sistema em camadas facilitou o trabalho em equipe, porque cada pessoa conseguiu testar o seu módulo sem esperar os outros.

- Definir o contrato antes da implementação (`IRepositorio<T>` e os cabeçalhos dos serviços) evita retrabalho.
- Os padrões de projeto não são enfeite: o Repository e a Factory resolveram problemas concretos, como esconder o SQL e recriar a subclasse certa a partir do banco.
- Testar com um banco em memória deixa os testes rápidos e repetíveis.
- Trabalhar com branches e Pull Requests ajuda a organizar o histórico e a conferir o código antes de entrar na `main`.
- Usar `unique_ptr` e `const&` evita vazamento de memória e cópias desnecessárias, sem precisar de `new` e `delete` no núcleo.

12. Referências

- SIMÕES, Francisco Paulo Magalhães. 7. *Project with OOP*. Slides da disciplina CIN0135 — Estruturas de Dados Orientadas a Objetos. Centro de Informática, UFPE, 2026.2.
- KIRCH-PRINZ, Ulla; PRINZ, Peter. *A Complete Guide to Programming in C++*. 2002.
- [Documentação do Qt](#)
- [Documentação do SQLite](#)
- [Documentação do CMake](#)
- [cppreference](#) — referência da linguagem C++
- Repositório do projeto: github.com/vivianazevedo/edoo-adm-condominio